WERKZEUGKARTE 1 Abi-Check – bevor du abgibst

Geh jede Implementierung mit dieser Liste durch. Jeder Haken ist eine gesicherte BE.

- ☐ **Signatur** genau wie gefordert (Name, Parameter, Rückgabetyp) – **Rückgabe in jedem Pfad**
- ☐ Vor jedem Zugriff: **Leer-, Index-, null- bzw. Blatt-Abfrage**
- ☐ **Schleifengrenzen:** Index von 0 bis Länge − 1 (Inventar: bis anzahl − 1)
- ☐ Struktur am Ende **unverändert**, falls die Aufgabe das fordert
- ☐ Nur **erlaubte Zeichenketten-Operationen** – Zeichenketten mit **equals** vergleichen
- ☐ Unsichere Stellen **kurz kommentieren** (`// Annahme: r ist nicht leer`)

Typische BE-Killer

- `return` fehlt in einem Zweig · `==` bei Zeichenketten · Schleife bis `length` statt `length - 1`
- Zugriff auf `null` (leerer Platz, fehlende Waffe) · private Attribute der Oberklasse direkt benutzt
- Unterklasse ohne `super(…)` als erste Anweisung im Konstruktor

✂ hier ausschneiden

WERKZEUGKARTE 2 Zeitbudget im Abitur

Faustregel: ca. 2,5 Minuten pro BE.

BE	Minuten	BE	Minuten
1	2,5	6	15
2	5	8	20
3	7,5	10	25
4	10	12	30
5	12,5	24	60

Anforderungsbereiche im Abitur

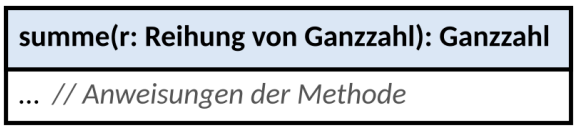
AFB I	AFB II	AFB III
30 %	53 %	17 %

- Erst alle Teilaufgaben überfliegen und die BE notieren.
- Zeit pro Teilaufgabe an den Rand schreiben.
- Hängst du fest: Lücke markieren, weitermachen, später zurückkommen.
- Am Ende 5 Minuten für den Abi-Check einplanen.

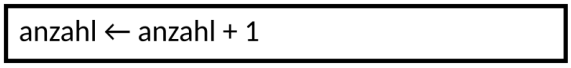
WERKZEUGKARTE 3 Struktogramm-Spickblatt (EH § 5)

- Sprachunabhängig. Zuweisung mit ←, Vergleiche mit =, ≠, <, ≤, >, ≥; Verknüpfung mit und, oder, nicht.
- Kopf mit Signatur `name(p: Typ): Typ`. **gib … zurück** beendet die Methode sofort. **erzeuge …** ruft einen Konstruktor auf.
- Zählschleife: „für i von 0 bis n − 1“ – **„bis“ ist einschließlich**. Kommentare mit `//`.

Kopf mit Signatur



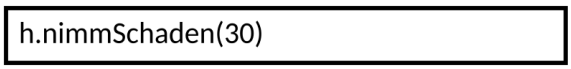
Zuweisung



Objekt erzeugen



Methodenaufruf



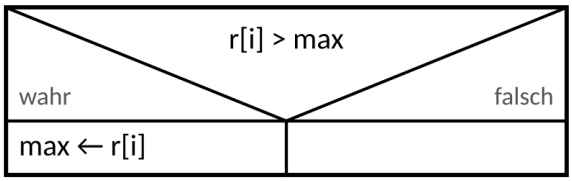
Rückgabe



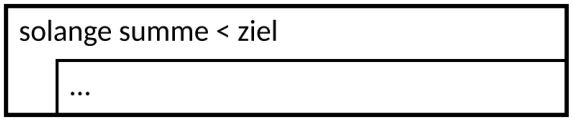
Kommentar



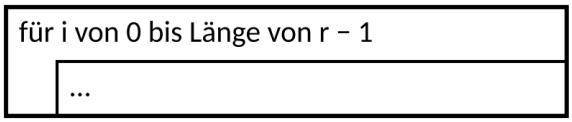
Verzweigung – links wahr, rechts falsch



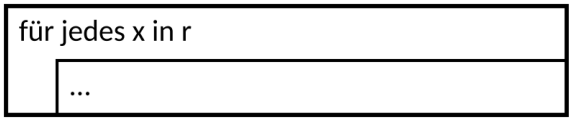
Solange-Schleife (kopfgesteuert)



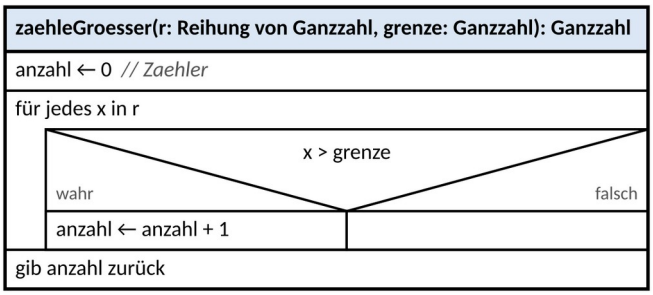
Zählschleife



Für-jedes-Schleife



Komplettes Beispiel




```
public static int zaehleGroesser(
        int[] r, int grenze) {
    int anzahl = 0;   // Zaehler
    for (int x : r) {
        if (x > grenze) {
            anzahl = anzahl + 1;
        }
    }
    return anzahl;
}
```

Typische Notationsfehler: Zuweisung mit = statt ← · leerer falsch-Zweig fehlt ganz · Schleifenrumpf nicht eingerückt · Kopf ohne Typen · „gib zurück“ vergessen.

WERKZEUGKARTE 4 Signaturen und Typnamen

name(parameter: Typ, …): Rückgabetyp z. B. `zaehle(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl`

Abitur	Java	Abitur	Java
Ganzzahl	`int`	nichts	`void`
Kommazahl	`double`	Reihung von Ganzzahl	`int[]`
Wahrheitswert	`boolean`	Reihung von Gegenstand	`Gegenstand[]`
Zeichen	`char`	Klassenname (z. B. Held)	`Held`
Zeichenkette	`String`	+ c Held(name: Zeichenkette)	`public Held(String name)`

- Immer **Reihung**, nie „Array“. Index 0 bis Länge − 1; 2D: `r[zeile][spalte]`. Länge auslesen und for-each sind erlaubt.
- `/` bei Ganzzahlen ganzzahlig (7 / 2 = 3), `%` liefert den Rest (7 % 2 = 1).
- Ganzzahlige Zufallszahl von 1 bis n: `(int) (Math.random() * n) + 1`. Ganzzahlig runden (x ≥ 0): `(int) (x + 0.5)`.

✂ hier ausschneiden

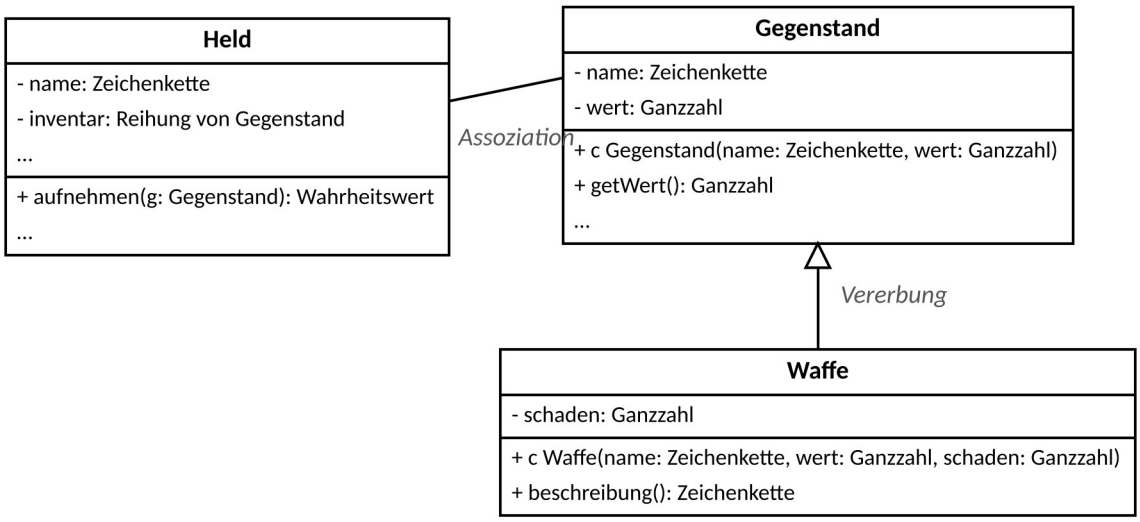
WERKZEUGKARTE 5 Erlaubte Zeichenketten-Operationen (abschließend!)

Operation	Java	Beispiel mit s = "Drache"
Länge	`s.length()`	6
Zeichen an Position	`s.charAt(i)`	`s.charAt(0)` → 'D'
Teilzeichenkette (a einschl., b ausschl.)	`s.substring(a, b)`	`s.substring(1, 4)` → "rac"
Verketten	`s + t`	`s + "nei"` → "Drachenei"
Gleichheit	`s.equals(t)`	`s.equals("Drache")` → true
enthält Teilzeichenkette	`s.contains(t)`	`s.contains("ach")` → true
lexikographischer Vergleich	`s.compareTo(t)`	< 0: s vorher · 0: gleich · > 0: s nachher
ASCII-Wert ↔ Zeichen	`(int) c` / `(char) n`	`(int) 'A'` → 65 · `(char) 97` → 'a'

Nicht erlaubt: `toLowerCase`, `toUpperCase`, `split`, `indexOf`, `trim`, `replace` … · Zeichenketten **nie** mit `==` vergleichen (die Online-IDE erlaubt es, das Abitur nicht). Einzelne Zeichen (char) dagegen mit `==`: `s.charAt(i) == 'a'`.

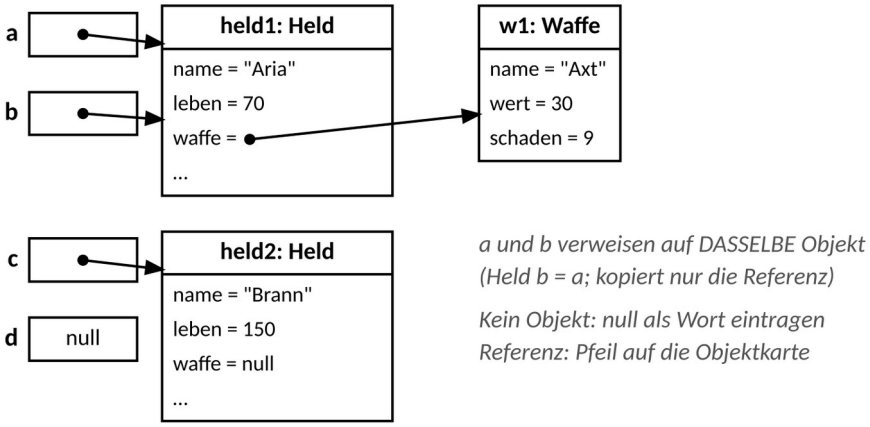
WERKZEUGKARTE 6 Klassendiagramm und Objektkarten (EH § 6)

Klassenkarten und Klassendiagramm



- Sichtbarkeit ist Pflicht: **–** privat (Attribute), **+** öffentlich (Methoden, Konstruktoren).
- Attribut `- leben: Ganzzahl` · Konstruktor `+ c Held(name: Zeichenkette)` · Methode `+ heilen(punkte: Ganzzahl): nichts`
- **Assoziation** (eine Klasse hat ein Attribut vom Typ der anderen, auch als Reihung): einfache Linie, **ohne** Beschriftung und Kardinalitäten.
- **Vererbung:** Pfeil mit **hohler Dreiecksspitze** zur Oberklasse; in der Unterklasse nur neue und überschriebene Elemente.

Objektkarten



a und b verweisen auf DASSELBE Objekt
(Held b = a; kopiert nur die Referenz)

Kein Objekt: null als Wort eintragen
Referenz: Pfeil auf die Objektkarte

- Kopf `objektname: Klasse`, darunter jedes Attribut mit konkretem Wert: `leben = 70`, Zeichenketten in Anführungszeichen.
- Referenzen als Pfeil auf die Objektkarte; kein Objekt: **null** als Wort. Reihungen als Kästchenreihe mit Index.
- `Held b = a;` kopiert nur die **Referenz** – es entsteht kein neues Objekt.

Kopiervorlage: Tracetabelle

Name: _____ Methode / Aufruf: _____

Eine Spalte je Variable (dazu ggf. Rückgabe/Ausgabe) – keine Bedingungen. Eine neue Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle im Code.

Schritt							
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							
20							
21							
22							
23							
24							

Kopiervorlage: Rekursionstabelle

Name: _____ Methode / erster Aufruf: _____

Eine Zeile je Aufruf. Tiefe 0 = erster Aufruf. Die Rückgabe trägst du ein, sobald der Aufruf endet – also von unten nach oben.

Aufruf (Nr.)	Parameter	Tiefe	Rückgabe	Bemerkung (z. B. Abbruchfall)
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				
22				

Kopiervorlage: Struktogramm-Raster

Name: _____ Aufgabe: _____ Kästchen = 5 mm

Signatur: